Сетевое взаимодействие контейнеров

Docker bridge, DNS, публикация портов и связь Express API с PostgreSQL

Цель занятия

- Объяснять, как Docker подключает контейнеры к сетям.
- Различать bridge, host и none и выбирать пользовательскую bridge-сеть.
- Понимать DNS-разрешение имён контейнеров и публикацию портов.
- Настраивать связь Express API с PostgreSQL по имени контейнера.
- Диагностировать доступность приложений и сетевую изоляцию.

На что опираемся

- ЛР №2: docker run, exec, ps, inspect и logs.
- ЛР №4: собственный Docker-образ Express API.
- ЛР №5: PostgreSQL 17 в контейнере pg-course и данные в томе pgdata-course.
- В этой теме соединяем готовые компоненты вручную; Docker Compose будет на следующем занятии.

Учебные вопросы

№	Вопрос	Основной результат
1	Сетевая модель Docker	Знать bridge, host, none
2	Пользовательская сеть и DNS	Соединять контейнеры по именам
3	Публикация портов	Отличать доступ с хоста от связи контейнеров
4	Express API + PostgreSQL	Организовать соединение через Docker-сеть
5	Диагностика и безопасность	Находить и исправлять сбои

Сетевая модель Docker

Контейнер — отдельное сетевое пространство имён, но не отдельный физический компьютер

Почему контейнерам нужна сеть

- Web API должен принимать HTTP-запросы от клиента.
- API необходимо соединяться с базой данных.
- Контейнеры могут работать отдельно, даже находясь на одном Docker-хосте.
- Сетевой драйвер Docker определяет способ подключения и изоляцию.

Network namespace в Linux

- У контейнера обычно собственные сетевые интерфейсы, IP-адреса, таблица маршрутизации и набор портов.
- Контейнер может обращаться к DNS, к другим контейнерам и во внешнюю сеть в пределах настроек.
- localhost внутри контейнера указывает на сам контейнер, а не на Linux-хост и не на соседний контейнер.

Docker-хост, bridge и сетевые интерфейсы контейнеров

Как Docker соединяет контейнеры внутри одного хоста

Docker-хост

Хостовая ОС (Linux)
Здесь работает Docker Engine
и сетевая подсистема

eth0 — Физический интерфейс хоста (доступ в сеть/Интернет)

Сетевой стек хоста
Маршрутизация, iptables, NAT, обработка трафика

docker0 (bridge)
Виртуальный мост Docker
Сеть: 172.17.0.0/16

Хост управляет сетью контейнеров

Bridge-сеть создаёт внутренний сегмент для контейнеров

docker0 связывает несколько контейнеров в одну внутреннюю сеть (L2)

docker0
(bridge)
172.17.0.0/16

Container A — IP: 172.17.0.2
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, веб-сервер)

Container B — IP: 172.17.0.3
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, БД)

Container C — IP: 172.17.0.4
veth (на хосте) — eth0 (в контейнере) — eth0 — Приложение (например, API)

Контейнер подключается к bridge через **veth**-пару

Что важно понять

1. У каждого контейнера есть собственный сетевой интерфейс eth0.
2. Контейнеры в одной bridge-сети могут обмениваться трафиком.
3. Для доступа с хоста обычно публикуют порт.
4. Внутри сети контейнеры не используют localhost друг друга.

Путь трафика

Приложение в контейнере → eth0 контейнера → veth → docker0 bridge → другой контейнер

Термины

Docker-хост	машина, где работает Docker Engine.
bridge	виртуальный коммутатор Docker.
eth0 контейнера	сетевой интерфейс внутри контейнера.
veth	виртуальная пара интерфейсов между контейнером и хостом.

Основные сетевые драйверы Docker

Драйвер	Принцип	Типичное использование
bridge	Виртуальная L2-сеть на одном Docker-хосте	Локальные многоконтейнерные приложения
host	Контейнер использует сетевое пространство имён хоста	Специальные сценарии без изоляции сети
none	Контейнер запускается без обычного сетевого интерфейса	Изолированные задачи без сети

Драйвер bridge

- Docker создаёт программный мост; контейнеры получают IP-адреса в выделенной подсети.
- Обычно включены правила сетевой изоляции и трансляции исходящих адресов.
- Для курса используем пользовательскую bridge-сеть на одном Docker-хосте.
- Overlay и межхостовая связь выходят за рамки этой лекции.

Драйвер host

- Контейнер разделяет сеть хоста вместо отдельного network namespace.
- Публикация портов через -p не применяется так же, как в bridge.
- Меньше изоляции сетевого уровня; для нашего проекта не требуется.

Драйвер none

- Сетевой стек контейнера максимально изолирован от внешних сетей; остаётся loopback.
- Подходит для вычислений без сетевого обмена.
- Для Express API и PostgreSQL не подходит: сервисам нужно взаимодействовать.

Просмотр имеющихся сетей

BASH

```
sudo docker network ls
```

Что важно

- Показывает NAME, DRIVER и SCOPE.
- Обычно присутствуют bridge, host и none.
- Имена и дополнительные пользовательские сети зависят от машины.

Пользовательская сеть и DNS

Почему имя контейнера удобнее динамического IP-адреса

Почему не стоит использовать IP контейнера в коде

- Контейнер может получить новый IP при пересоздании.
- Жёстко заданный IP ухудшает переносимость и усложняет поддержку.
- В пользовательской bridge-сети Docker предоставляет встроенное DNS-разрешение имён контейнеров и aliases.

Стандартная bridge и пользовательская bridge

Стандартная bridge

- Подключение по умолчанию без --network.
- Автоматическое DNS-разрешение по именам контейнеров не предоставляется обычным образом.
- Общая сеть для неявно подключённых контейнеров.

Пользовательская bridge

- Создаётся docker network create.
- Встроенное DNS-разрешение по имени контейнера.
- Легко отделить учебное приложение от посторонних контейнеров.

Создать сеть course-net

BASH

```
sudo docker network create --driver bridge course-net
sudo docker network inspect course-net
```

Что важно

- course-net — имя учебной сети.
- Драйвер bridge обеспечивает связь на одном Docker-хосте.
- inspect показывает подсеть, шлюз и подключённые контейнеры.

Подключить существующий контейнер

BASH

```
sudo docker network connect course-net pg-course
sudo docker network inspect course-net
```

Что важно

- Команда подключает уже работающий pg-course к сети course-net.
- Контейнер не пересоздаётся; данные тома сохраняются.
- Если сеть уже подключена, команда сообщит об этом; проверьте inspect.

Пользовательская bridge-сеть и DNS имена контейнеров

Как контейнеры находят друг друга по имени внутри Docker-сети

Docker-хост

Docker Engine
Управляет контейнерами, сетями и DNS

Пользовательская сеть (bridge)

course-net
(bridge)

- Отдельная пользовательская bridge-сеть
- Изолирована от других сетей
- Контейнеры в этой сети могут общаться по именам

Пользовательская Docker-сеть: course-net (bridge)

Сеть course-net
Подсеть: 172.20.0.0/16

api-net6
Express API
IP: 172.20.0.2

api-net6 → pg-course:5432

Обращение к базе данных по имени хоста **"pg-course"** (а не по localhost)

pg-course
PostgreSQL
IP: 172.20.0.3

Может обращаться по имени

client-test
Временный тестовый контейнер
IP: 172.20.0.4

Может обращаться по имени

Как работает Docker DNS

Встроенный DNS автоматически разрешает имена контейнеров в IP-адреса в сети course-net.

1 Контейнер отправляет запрос к имени "pg-course"

api-net6 → Запрос: "pg-course" → Docker DNS (встроенный)

2 Docker DNS находит IP контейнера

В сети course-net есть контейнер с именем **"pg-course"**.

pg-course
172.20.0.3

3 Устанавливается соединение

api-net6 → Соединение по IP 172.20.0.3:5432 → pg-course (PostgreSQL)

Правильно

Использовать имя контейнера как host (например, "pg-course").

```
host=pg-course
```

- Контейнеры общаются по имени в одной сети (course-net).
- Docker DNS автоматически разрешает имя в IP-адрес.

Неправильно

Использовать `localhost` для связи между контейнерами.

```
host=localhost
```

- `localhost` внутри контейнера указывает на сам контейнер.
- Контейнер api-net6 не сможет подключиться к базе данных, используя localhost.

Пример подключения

Настройки приложения для подключения к PostgreSQL:

```
DB_HOST=pg-course
DB_PORT=5432
DB_NAME=shop
DB_USER=postgres
```

Эти настройки можно использовать в переменных окружения приложения (api-net6).

Что важно понять

1. В пользовательской bridge-сети контейнеры получают DNS-имена.
2. Имя контейнера можно использовать как host в приложении.
3. Для связи контейнеров публикация порта не обязательна.
4. Порт публикуют только для доступа с хоста (например, чтобы открыть API или базу данных в браузере или внешних клиентах).

Встроенный DNS на пользовательской сети

- Express API сможет обращаться к PostgreSQL по имени pg-course.
- Имя резолвится только в сетевом контексте, где доступен соответствующий контейнер.
- DNS отвечает за преобразование имени в адрес, а не за доступность СУБД или проверку пароля.

Быстрая проверка DNS по имени

BASH

```
sudo docker run --rm --network course-net busybox:1.3
```

Что важно

- Контейнер busybox запускается временно и удаляется после завершения.
- Ожидается ответ с адресом pg-course в сети course-net.
- Если имя не найдено, проверьте docker network inspect и состояние базы данных.

Публикация портов

Путь клиента к API отличается от связи API с базой данных

Контейнерный порт и порт хоста

- Приложение внутри контейнера прослушивает свой порт — например, 3000.
- Клиент с хоста обращается к опубликованному порту Docker-хоста.
- Контейнеры в одной пользовательской bridge-сети могут обращаться друг к другу по внутреннему порту без публикации -p.

Публикация порта API

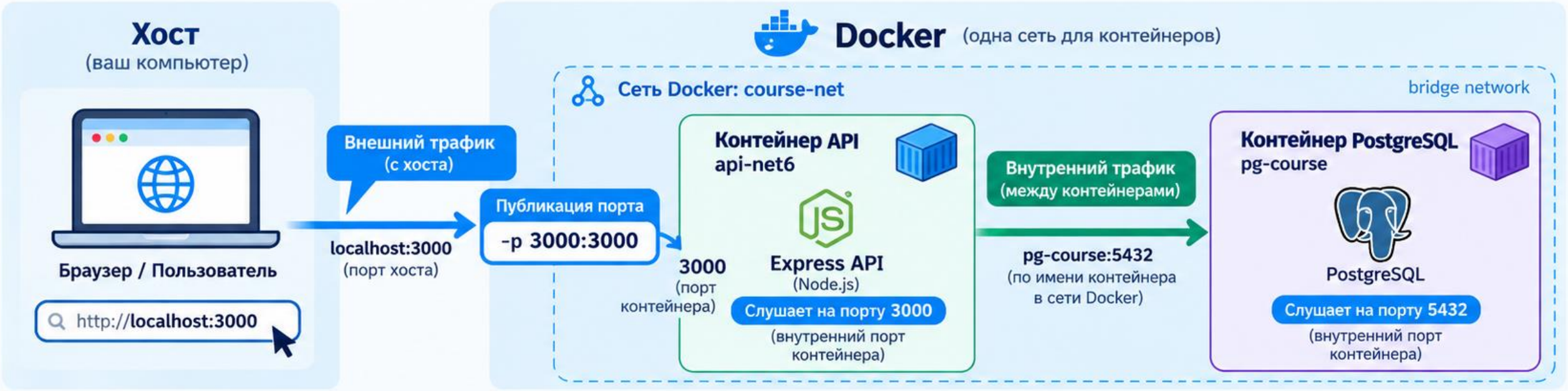
```bash
sudo docker run -d --name api-net6 --network course-
```

Что важно

- Слева указан адрес/порт Linux-хоста, справа — порт контейнера.
- 127.0.0.1 ограничивает публикацию локальными подключениями к хосту.
- Для внешнего доступа из Windows к Debian-ВМ понадобится иная настройка адреса/маршрута и firewall.

Порт хоста, порт контейнера и внутренний трафик

Как работает публикация порта в Docker и чем внешний доступ отличается от связи между контейнерами



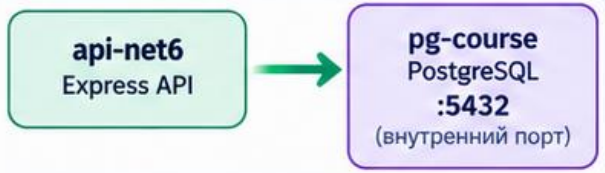
Внешний доступ с хоста

Браузер → localhost:3000 → api-net6:3000



Внутренний трафик между контейнерами

api-net6 → pg-course:5432 (в сети Docker)



Трафик идёт по имени контейнера в Docker-сети и не использует опубликованный порт хоста.

Правильно / Неправильно

Правильно:

- Хост обращается к API через `http://localhost:3000`
- API обращается к БД через `pg-course:5432`

Неправильно:

- API обращается к БД через `localhost:5432`

Внутри контейнера localhost — это сам контейнер, а не контейнер с БД.

Пример команды запуска

```
docker run -d --name api-net6 \
  --network course-net \
  -p 3000:3000 \
  api-net6:latest
```

Настройки приложения (например, .env)

```
DB_HOST=pg-course
DB_PORT=5432
```

Что важно понять

1. Публикация порта открывает доступ с хоста к контейнеру.
2. Контейнер продолжает слушать свой внутренний порт.
3. Связь между контейнерами в одной сети идёт по именам и внутренним портам.
4. Host port и container port — это не одно и то же.

Какую настройку использовать?

Направление	Адрес	Нужен -p?
Браузер Debian → API	http://127.0.0.1:3000	Да, для bridge
Контейнер API → PostgreSQL	pg-course:5432	Нет
API → localhost:5432 внутри API	Не PostgreSQL	Нет, неверный адрес

EXPOSE и публикация портов

- EXPOSE 3000 в Dockerfile описывает ожидаемый порт приложения.
- EXPOSE не создаёт доступ с хоста автоматически.
- Команда docker run -p публикует порт; если привязка не указана, Docker обычно публикует его на всех интерфейсах хоста.
- Для базы данных не публикуем 5432 на хост без необходимости.

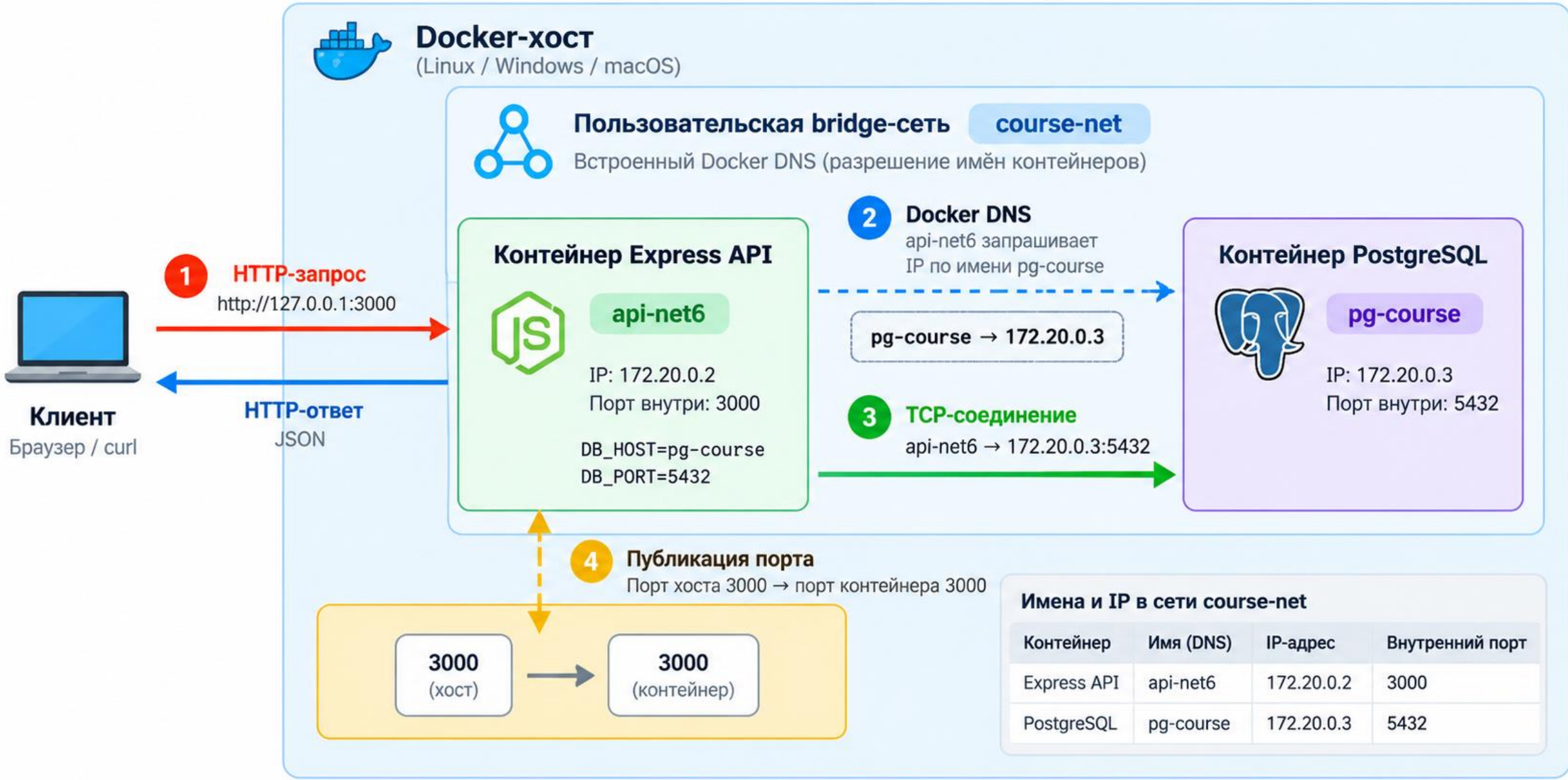
localhost: главное правило

- localhost в браузере Debian — сама Debian.
- localhost внутри API-контейнера — API-контейнер.
- Для подключения к PostgreSQL используем PGHOST=pg-course.
- Определите, откуда выполняется запрос, прежде чем выбирать адрес.

Express API и PostgreSQL

Связываем два контейнера из сквозного проекта

Express API → Docker DNS → PostgreSQL



Имена и IP в сети course-net

Контейнер	Имя (DNS)	IP-адрес	Внутренний порт
Express API	api-net6	172.20.0.2	3000
PostgreSQL	pg-course	172.20.0.3	5432

Подготовленные компоненты

- Контейнер pg-course из лабораторной №5 работает с БД shop, пользователем student и отдельным томом.
- Собственный Express API из лабораторной №4 расширим пакетом pg для SQL-запросов.
- Оба контейнера подключим к сети course-net; API опубликуем на 127.0.0.1:3000.
- Это демонстрационная конфигурация, без Docker Compose и без публикации PostgreSQL.

Минимальное соединение Node.js с PostgreSQL

```javascript
import pg from "pg";
const { Pool } = pg;

const pool = new Pool({
  host: process.env.PGHOST,
  port: 5432,
  user: process.env.PGUSER,
  password: process.env.PGPASSWORD,
  database: process.env.PGDATABASE
});

export default pool;
```

Что важно

- pg — клиент PostgreSQL для Node.js; Pool управляет соединениями.
- PGHOST=pg-course: адрес соседнего контейнера по имени.
- Пароль передаётся через переменную окружения только в учебной среде; продакшену нужны более безопасные механизмы.

Маршрут диагностики базы данных

```javascript
app.get("/api/db-check", async (req, res) => {
  try {
    const result = await pool.query("SELECT NOW() AS
    res.json({ ok: true, serverTime: result.rows[0].s
  } catch (error) {
    console.error(error);
    res.status(503).json({ message: "База данных недо
  }
});
```

Что важно

- GET-запрос проверяет реальное SQL-соединение, а не только DNS.
- При успехе возвращается JSON с временем сервера БД.
- При недоступности БД API возвращает 503 без раскрытия пароля.

Последовательность запуска компонентов

1. **Шаг 1** — Проверить, что pg-course запущен и данные лабораторной №5 сохранились.
2. **Шаг 2** — Создать course-net и подключить pg-course.
3. **Шаг 3** — Добавить пакет pg и маршрут /api/db-check в Express-проект.
4. **Шаг 4** — Собрать express-pg:net6 и запустить api-net6 в сети course-net.
5. **Шаг 5** — Выполнить curl http://127.0.0.1:3000/api/db-check.
6. **Шаг 6** — Проверить подключённые контейнеры в docker network inspect course-net.

Что произойдёт при остановке БД

- DNS-имя может разрешаться, но PostgreSQL перестанет принимать соединения.
- API должен обработать отказ и вернуть понятный HTTP-ответ.
- После docker start pg-course и готовности БД запрос снова сможет выполниться.
- Данные остаются в томе pgdata-course.

Диагностика и безопасность

Проверяем DNS, TCP-подключение, HTTP и SQL отдельно

Проверить сеть и контейнеры

BASH

```
sudo docker network ls
sudo docker network inspect course-net
sudo docker ps
sudo docker inspect api-net6
```

Что важно

- Сначала проверьте, к одной ли сети подключены pg-course и api-net6.
- Убедитесь, что контейнеры имеют состояние Up.
- Проверьте NetworkSettings, PortBindings и окружение без публикации паролей.

Проверить API и журналы

BASH

```
curl -i http://127.0.0.1:3000/api/db-check
sudo docker logs api-net6
sudo docker logs pg-course
```

Что важно

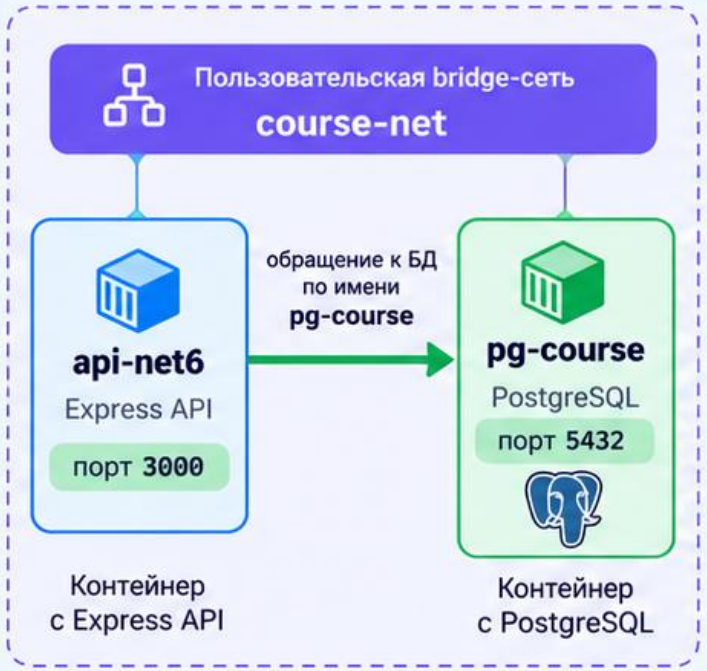
- Ожидается HTTP 200 и JSON с ok: true.
- При HTTP 503 анализируйте сеть, готовность PostgreSQL и настройки pg.
- Не выводите значения паролей в отчёт.

Диагностика связи API и PostgreSQL

Как проверить сетевое соединение между контейнером Express API и контейнером PostgreSQL

Среда

Docker Host
(ваш компьютер / сервер)



Пошаговая диагностика

1 Проверить, что оба контейнера запущены

Убедимся, что контейнеры api-net6 и pg-course работают.

```
$ docker ps
$ docker inspect api-net6
$ docker inspect pg-course
```

✓ Контейнеры должны иметь статус `Up`.

2 Проверить подключение к сети

Проверим, что контейнеры находятся в одной сети course-net.

```
$ docker network inspect course-net
```

✓ Оба контейнера должны присутствовать в сети `course-net`.

3 Проверить DNS-имя контейнера БД

Проверим, что из контейнера API имя pg-course разрешается.

```
$ docker exec -it api-net6 sh
$ ping pg-course
```

✓ Имя `pg-course` должно разрешаться во внутренний IP.

4 Проверить доступность порта PostgreSQL

Проверим, что порт 5432 доступен из контейнера API.

```
$ nc -zv pg-course 5432
```

или использовать клиент `psql` / приложение.

✓ Порт 5432 должен быть доступен внутри сети Docker.

5 Проверить приложение

Выполним HTTP-запрос к маршруту, который проверяет соединение с БД.

```
$ curl http://127.0.0.1:3000/api/db-check
{
  "ok": true,
  "categories": 2
}
```

✓ Маршрут API подтверждает соединение с PostgreSQL.

Типичные ошибки

Контейнер PostgreSQL остановлен
Контейнер `pg-course` не запущен или перезапускается с ошибкой.

Контейнеры не в одной сети
Контейнеры подключены к разным сетям или один из них использует сеть по умолчанию (`bridge`).

Вместо pg-course используется *localhost*
Внутри контейнера API нельзя использовать localhost для доступа к другому контейнеру.

Неверные DB_HOST / DB_PORT
В переменных окружения указаны неправильные значения (например, `localhost` вместо `pg-course` или неверный порт).

PostgreSQL ещё не готов принимать соединения
Контейнер запущен, но база данных ещё инициализируется.
Нужно подождать или настроить healthcheck / retry.

Что важно понять

1. Для связи контейнеров нужна общая Docker-сеть.
2. Внутри сети Docker используют имя контейнера, а не localhost.
3. Публикация порта нужна для доступа с хоста, но не для связы контейнеров между собой.
4. Диагностику удобнее выполнять по цепочке: контейнеры → сеть → DNS → порт → приложение.

Типовые ошибки

Признак	Возможная причина	Что проверить
ENOTFOUND pg-course	Нет общей сети или имя неверно	network inspect, PGHOST
ECONNREFUSED	БД не слушает порт/не запущена	docker ps, pg_isready, logs
28P01 authentication failed	Неверный пароль или пользователь	PGUSER и PGPASSWORD
HTTP не открывается на хосте	Порт не опубликован или процесс не слушает	docker ps, -p, logs

Обратимый эксперимент

- Остановите pg-course; запрос /api/db-check должен временно перестать работать.
- Запустите pg-course, дождитесь готовности по pg_isready и повторите HTTP-запрос.
- В ходе эксперимента не удаляйте том pgdata-course.

Сетевая безопасность учебного проекта

- Не публикуйте PostgreSQL на все интерфейсы без необходимости.
- Порт API можно привязать к 127.0.0.1 при проверке на хосте Debian.
- Пароли не следует сохранять в образ Dockerfile и публичном репозитории.
- При запуске на общем сервере пароль и доступ к сети следует настраивать отдельно.

Итоги лекции

1. Пользовательская bridge-сеть соединяет контейнеры на одном Docker-хосте.
2. Встроенный Docker DNS позволяет обращаться к контейнерам по имени, например pg-course.
3. localhost внутри контейнера не указывает на другой контейнер.
4. -p публикует порт для клиентов вне внутренней сети; связь API → БД обходится без -p для PostgreSQL.
5. DNS, транспорт, аутентификацию и SQL нужно диагностировать как отдельные уровни.

Контрольные вопросы

1. 1. В чём отличие bridge, host и none?
2. 2. Почему для приложения удобнее имя pg-course, чем IP контейнера?
3. 3. Что означает -p 127.0.0.1:3000:3000?
4. 4. Почему API нельзя подключать к БД через localhost:5432?
5. 5. Что показывает docker network inspect?
6. 6. Чем проверка DNS отличается от проверки SQL-соединения?
7. 7. Для чего нужна команда docker network connect?
8. 8. Почему PostgreSQL не обязательно публиковать на хост?
9. 9. Что произойдёт с данными тома при остановке PostgreSQL?
10. 10. Какие шаги предпринять при ошибке ENOTFOUND?